

24: Caches

Memory Hierarchy (Recap)

- Registers
- SRAM
- DRAM
- Disk (flash)

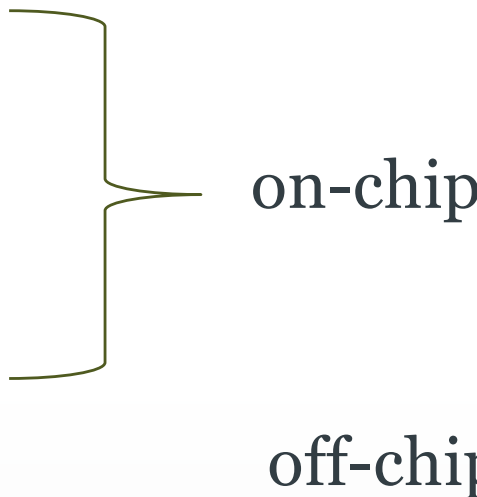


Increasing cost/bit
Increasing speed

Note that Static RAM (SRAM) is CMOS; Dynamic RAM (DRAM) is a different technology. Can't be integrated on the same chip.

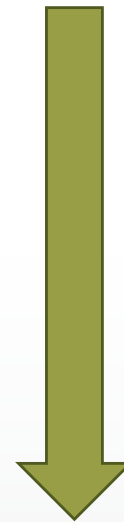
Without on-chip caches, processors would stall because DRAM is slow. Fastest read time is $\sim 20\text{ns}$ (50 MHz).

Multi-Level Caches

- L1 I + D
 - L2
 - L3
 - L4
- 
- The diagram consists of a vertical line with a bracket on the right side. The bracket spans the first three items of the list (L1 I + D, L2, and L3) and is labeled 'on-chip'. Below this, the fourth item (L4) is positioned to the left of the text 'off-chip'.
- Intel i7 – L1 32k I + D; L2 256k; L3 8M
 - Each bit of SRAM needs 6 transistors ~400M transistors for L3. (L3 cache has ~40 cycle latency.)

Associativity

- Where in the cache can a block of data go?
 - Direct map (one place)
 - 2-way set associativity
 - 4-way set associativity
 - ...
 - Fully associative (anywhere)
- Can be achieved by masking bits (but gets more complex)

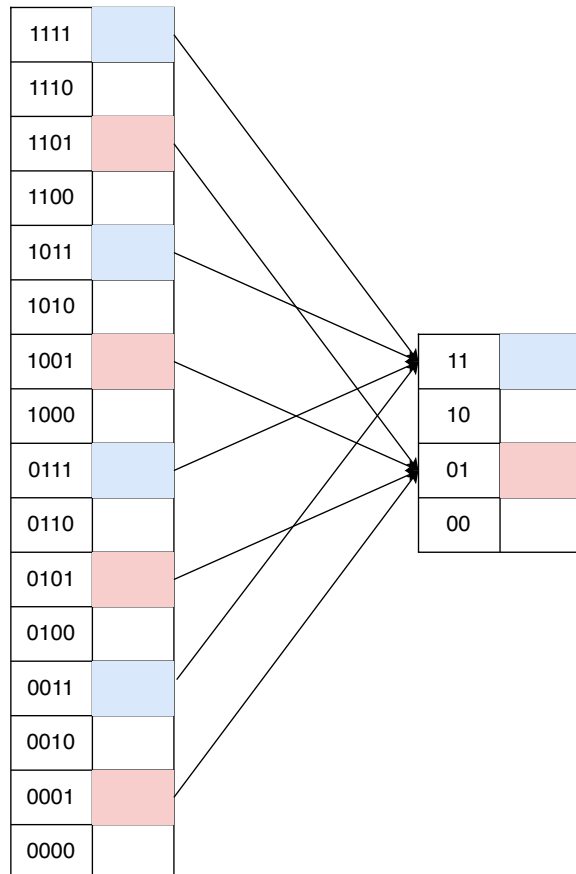


Better but
more complex

Principle of Locality

- Try to make all memory *appear* fast and local
 - Not possible to achieve in reality
- Cache miss – instruction/data not in cache
- Cache hit – can immediately read/write

Direct-Mapped Cache



- 16 word memory
- 4 word cache
- Each memory location maps to exactly one cache location

Direct-Mapped Cache

- Simple Example
 - 16 word cache (4 bits for the cache address)
 - 64 word memory (6 bits)
- Cache address = (memory address) modulo (number of addresses in the cache)
- Let's see how the following sequence of read addresses is mapped to the cache:
 - 1, 4, 8, 5, 20, 17, 19, 56, 9, 11, 33, 43, 5, 1, 9, 17
 - Contents column indicates the contents of the address shown memory(1), etc.

Address	HIT/MISS	
1	MISS	
4	MISS	
8	MISS	
5	MISS	
20	MISS	$20\%16 = 4$
17	MISS	$17\%16 = 1$
19	MISS	$19\%16 = 3$
56	MISS	$56\%16 = 8$
9	MISS	
11	MISS	
33	MISS	$33\%16 = 1$
43	MISS	$43\%16 = 11$
5	HIT	
1	MISS	
9	HIT	
17	MISS	

Cache Address	Contents of memory
0	
1	1 , 17 , 33 , 1 , 17
2	
3	19
4	4 , 20
5	5
6	
7	
8	8 , 56
9	9
10	
11	11 , 43
12	
13	
14	
15	

Tags and Valid bit

- How do we know whether we have a hit or a miss? How do we know the real address?
 - Include a Valid bit in the cache
 - Include a tag (the top bits of the address)
- Flush the cache by setting all the Valid bits to zero
 - Cache will still contain data – possible security risk
- Cache after first four reads (next slide)

Cache Address	Valid	Tag	Contents
0	0		
1	1	00	1
2	0		
3	0		
4	1	00	4
5	1	00	5
6	0		
7	0		
8	1	00	8
9	0		
10	0		
11	0		
12	0		
13	0		
14	0		
15	0		

Tags

- Address 1 is (6 bits) 000001, so tag is 00 (top 2 bits)
 - Similarly for addresses 4, 8, 5
- Address 20 is 010100, so tag is 01
- Cache becomes:

Cache Address	Valid	Tag	Contents
0	0		
1	1	00	1
2	0		
3	0		
4	1	01	20
5	1	00	5
6	0		
7	0		
8	1	00	8
9	0		
10	0		
11	0		
12	0		
13	0		
14	0		
15	0		

Cache size

- Cache size is usually stated in terms of the number of bits of data stored
 - In this example, 16 addresses x data size, say, 16 bits
- But cache also has to store tag and valid bits
 - Here, $16 \times (2 + 1) = 48$ bits
- This adds significantly to the overall size of the cache
 - Could be greater than the data storage size

Block Size

- Previous example assumes that the cache fetches one word at a time
 - If we are loading instructions, we need 4 words for each instruction
 - It is very likely that we will need a sequence of instructions
 - Similarly, data is likely to be grouped in memory
- May be more efficient to load blocks of more than one word
 - Previous example, with 4 word blocks

Address	HIT/MISS	
1	MISS	
4	MISS	
8	MISS	
5	HIT	
20	MISS	$20\%16 = 4$
17	MISS	$17\%16 = 1$
19	HIT	$19\%16 = 3$
56	MISS	$56\%16 = 8$
9	MISS	
11	HIT	
33	MISS	$33\%16 = 1$
43	MISS	$43\%16 = 11$
5	MISS	
1	MISS	
9	MISS	
17	MISS	

Cache Address	Contents of memory
0	0, 16, 32, 0, 16
1	1, 17, 33, 1, 17
2	2, 18, 34, 2, 18
3	3, 19, 35, 3, 19
4	4, 20, 4
5	5, 21, 5
6	6, 22, 6
7	7, 23, 7
8	8, 56, 8, 40, 8
9	9, 57, 9, 41, 9
10	10, 58, 10, 42, 10
11	11, 59, 11, 43, 11
12	
13	
14	
15	

Cache Misses

- Handling a Cache Hit is simple – everything carries on, normally
- A Cache Miss means a stall (not an exception or interrupt)
- Cache controller fetches data from memory, or next level cache. Meanwhile the processor waits.
- Note that an instruction cache miss means that the content of the instruction register is incorrect, so the instruction must be reloaded when the cache load is complete

Cache Writing

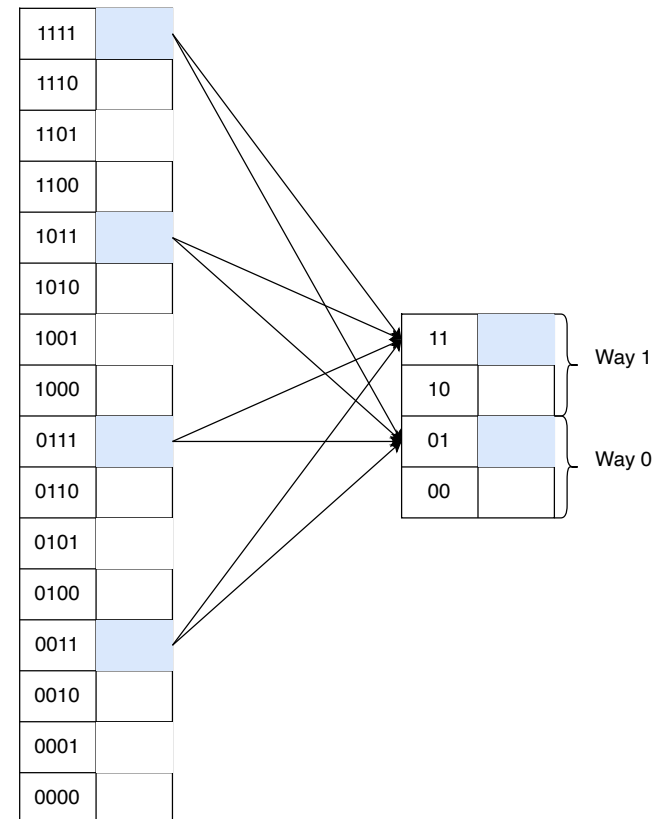
- Need to ensure that data in memory and copy in cache are consistent
- Two approaches:
 - Write-through – write data "through" caches back to memory. Slow (can cause stalls), but accurate.
 - Write-back – mark changed data with a "dirty" bit. Write back only when cache is flushed.
- Write-through is simpler. Can also buffer writes to (try to) avoid stalls.

Associativity

- Direct Mapping means that a block can only go in exactly one place in the cache
- A Fully Associative cache allows data to be placed anywhere
- A Set Associative cache has a fixed number of possible locations
- Need a cache replacement strategy
 - Least Recently Used is common

Two-way associative cache

- Same as previous example, but data can go to one of two places
 - For example, data at 3, 7, 11, 15 can go to address 1 in way 1 or way 0 (11 or 10)
 - When evicting data, assume Least Recently Used



Address	HIT/MISS	
1	MISS	
4	MISS	
8	MISS	
5	MISS	
20	MISS	$20\%8 = 4$
17	MISS	$17\%8 = 1$
19	MISS	$19\%8 = 3$
56	MISS	$56\%8 = 0$
9	MISS	$9\%8 = 1$
11	MISS	$11\%8 = 3$
33	MISS	$33\%8 = 1$
43	MISS	$43\%8 = 3$
5	HIT	
1	MISS	
9	MISS	
17	MISS	

Cache Address	Way	Contents
0	0	8
1	0	1, 9, 1, 17
2	0	11
3	0	43
4	0	4
5	0	5
6	0	
7	0	
8	1	56
9	1	17, 33, 9
10	1	
11	1	19
12	1	20
13	1	
14	1	
15	1	

Fully associative cache

- Same as previous example, but data can go anywhere
 - 16 1-word blocks
 - For simplicity, do this in the order shown

Address	HIT/MISS
1	MISS
4	MISS
8	MISS
5	MISS
20	MISS
17	MISS
19	MISS
56	MISS
9	MISS
11	MISS
33	MISS
43	MISS
5	HIT
1	HIT
9	HIT
17	HIT

Cache Address	Contents
0	1
1	4
2	8
3	5
4	20
5	17
6	19
7	56
8	9
9	11
10	33
11	43
12	
13	
14	
15	

Fully associative cache with blocks

- Same as previous example, but data can go anywhere
 - 4 4-word blocks
 - For simplicity, do this in the order shown, but align addresses

Address	HIT/MISS
1	MISS
4	MISS
8	MISS
5	HIT
20	MISS
17	MISS
19	HIT
56	MISS
9	HIT
11	HIT
33	MISS
43	MISS
5	MISS
1	MISS
9	MISS
17	MISS

Cache Address	Contents of memory
0	0, 16, 40, 16
1	1, 17, 41, 17
2	2, 18, 42, 18
3	3, 19, 43, 19
4	4, 56, 4
5	5, 57, 5
6	6, 58, 6
7	7, 59, 7
8	8, 0
9	9, 1
10	10, 2
11	11, 3
12	20, 32, 8
13	21, 33, 9
14	22, 34, 10
15	23, 35, 11

Associativity – Summary

- Direct-mapped – 2 hits
- Direct-mapped, 4 blocks – 3 hits
- Two-way associative – 1 hit
- Fully associative – 4 hits
- Fully associative, 4 blocks – 4 hits
- Note that the cache design is becoming more complex
- (Not exactly a rigorous analysis!)

Victim Cache

- When replacing instructions/data in cache, can move old instructions/data out to the next level (rather than discard)
- Known as a "victim cache"
- When reading in instructions/data, usually read in a block. Idea is that sections of code are located together and execute together. Same for data.
 - Fails if, for example, a loop is just bigger than the cache
 - Hence a victim cache can help to bring code back to L1

Victim Cache

- Assume small, 4-word, fully associative victim cache. How would this help in each case?
- Direct mapped: last MISS, 17, will still be in the Victim cache

Address	HIT/MISS	Eviction
1	MISS	
4	MISS	
8	MISS	
5	MISS	
20	MISS	4
17	MISS	1
19	MISS	3
56	MISS	8
9	MISS	
11	MISS	
33	MISS	17
43	MISS	11
5	HIT	
1	MISS	33
9	HIT	
17	Victim cache	1

Cache Address	Contents of memory
0	
1	1 , 17 , 33 , 1 , 17
2	
3	19
4	4 , 20
5	5
6	
7	
8	8 , 56
9	9
10	
11	11 , 43
12	
13	
14	
15	

Two-way associative with victim cache

- Last three evictions will be in the victim cache
- Note that in these cases, the LRU policy does not affect the place for restoring from the victim cache ... but it might

Address	HIT/MISS	Eviction
1	MISS	
4	MISS	
8	MISS	
5	MISS	
20	MISS	
17	MISS	
19	MISS	
56	MISS	
9	MISS	1
11	MISS	
33	MISS	17
43	MISS	
5	HIT	
1	Victim Cache	9
9	Victim Cache	33
17	Victim Cache	1

Cache Address	Way	Contents
0	0	8
1	0	1, 9, 1, 17
2	0	11
3	0	43
4	0	4
5	0	5
6	0	
7	0	
8	1	56
9	1	17, 33, 9
10	1	
11	1	19
12	1	20
13	1	
14	1	
15	1	

For you to do

- The following is a sequence of word addresses: 1, 5, 8, 23, 11, 25, 7, 8, 33, 49, 11, 7, 8, 1, 33, 11. Show whether a read to each of these addresses is a hit or miss for each of the following cases. Assume the cache is initially empty. Assume Least Recently Used (LRU) replacement. Show the final contents of the cache.
 - i. Direct mapped cache with 16 one-word blocks.
 - ii. Direct mapped cache with 4 four-word blocks. (The total size is 16 words; 4 words are loaded at once.)
 - iii. Two-way associative cache with 16 one-word blocks.
 - iv. Two-way associative cache with 4 four-word blocks.
 - v. Fully associative cache with 16 one-word blocks.
 - vi. Fully associative cache with 4 four-word blocks.